

BASE Properties

BASE = Basically Available, Soft state, Eventually consistent

Basically Available - The system guarantees availability — responses are provided even if they're not guaranteed to be the most recent data. The system appears to work all the time.

Trade-off:

- Availability over consistency
- Always responds, but data might be stale
- Better user experience (no waiting/timeouts)

```
**Example - Distributed Database:**  
...
```

```
User in New York writes to database  
User in London reads immediately after
```

ACID system:

- Waits for New York → London replication
- User might experience delay or timeout
- OR transaction is rejected to maintain consistency

BASE system:

- Returns immediately with whatever data London has
 - Might be slightly stale (hasn't replicated yet)
 - System stays available
- ```
...
```

**Soft State** - The state of the system may change over time, even without input, due to eventual consistency. Data might be in flux during propagation.

```
Example - Social Media Post:
...
```

```
Time 0:00 You post "Hello World"
Time 0:01 Friend in USA sees the post (fast replication)
Time 0:03 Friend in Australia doesn't see it yet (slow replication)
Time 0:05 Friend in Australia now sees it (caught up)
```

```
The "state" changed from invisible → visible without new input
System was in "soft state" during replication
...
```

```
Contrast with ACID:
...
```

```
ACID: State only changes with explicit transactions
Before commit: old state
After commit: new state (instant, everywhere)
```

```
BASE: State gradually changes as updates propagate
Intermediate states exist
System "soft" until converges
...
```

**Eventually Consistent** - If no new updates are made, eventually all replicas will converge to the same value. Consistency is not immediate but is guaranteed over time.

**\*\*Example - Distributed Counter (page views):\*\***

...

Article **view** count across **3 data** centers:

| Time | DC1 | DC2 | DC3 | Actual Total         |
|------|-----|-----|-----|----------------------|
| 0:00 | 100 | 100 | 100 | 100                  |
| 0:01 | 101 | 100 | 100 | (user viewed in DC1) |
| 0:02 | 101 | 100 | 100 | (replicating...)     |
| 0:03 | 101 | 101 | 100 | (DC2 updated)        |
| 0:04 | 101 | 101 | 101 | (DC3 updated)        |
| 0:05 | 101 | 101 | 101 | <b>Consistent!</b> ✓ |

Given **no** more updates, **all** converged to **101**

...

## Types of Eventual Consistency

### 1. Casual Consistency

- Operations that are causally related are seen in order.

User A posts: "What's for dinner?"

User B replies: "Pizza!"

Everyone sees question before answer

**Order** preserved **for** related operations

...

### 2. Read-your-writes Consistency

- A user always sees their own updates.

You **update** your profile picture

You immediately see new picture

Others might see old picture briefly

But **YOU** always see your own changes

...

### 3. Monotonic Read Consistency

- Once you read a value, you never see an older value.

### 4. Monotonic Write Consistency

- Your writes are applied in order.

You **write**: balance = \$**100**

**Then write**: balance = \$**150**

These **apply in order** at **all** replicas

Never reversed

...

## BASE System - Social Media Feed

javascript

```
// User posts update
async function createPost(userId, content) {
 // Write to primary database (US-East)
 const post = await db.usEast.posts.create({
 userId: userId,
 content: content,
 timestamp: Date.now()
 });

 // Immediately return success to user
 // Don't wait for replication
 return { success: true, postId: post.id };

 // Background: Asynchronously replicate to other regions
 // US-West, EU, Asia regions will get update eventually
 backgroundReplication(post);
}

// User views feed
async function getFeed(userId, region) {
 // Read from nearest database
 const posts = await db[region].posts.find({
 following: userId
 }).sort({ timestamp: -1 }).limit(50);

 // Might not include post from 1 second ago
 // That's okay! Eventually it will appear
 return posts;
}

// Properties demonstrated:
// Basically Available: Always returns feed (even if slightly stale)
// Soft State: Feed changes as replication happens
// Eventual Consistency: Given time, all regions see same posts
```

### Why BASE works:

- Seeing friend's post 2 seconds late is acceptable
- High availability more important than perfect consistency
- Global scale requires accepting replication delay
- User experience benefits from fast responses

## ACID vs BASE Comparison

| Aspect              | ACID                             | BASE                                        |
|---------------------|----------------------------------|---------------------------------------------|
| Consistency         | Strong, immediate                | Eventual, delayed                           |
| Availability        | May sacrifice for consistency    | Prioritizes availability                    |
| Partition Tolerance | Weaker                           | Stronger                                    |
| Performance         | Slower (coordination overhead)   | Faster (less coordination)                  |
| Scalability         | Harder to scale horizontally     | Easier to scale horizontally                |
| Data Accuracy       | Always accurate                  | Temporarily inaccurate, eventually accurate |
| Use Cases           | Banking, inventory, reservations | Social media, analytics, caching            |
| Databases           | PostgreSQL, MySQL, Oracle        | Cassandra, DynamoDB, MongoDB                |
| User Experience     | Might see delays/errors          | Snappier, always available                  |
| Complexity          | Simpler application logic        | Complex conflict resolution                 |

### When to Choose BASE

**High availability critical**

**Scale over consistency**

**Eventual consistency acceptable**

- Social media feeds
- View counts, likes, shares
- Product catalog (prices can be slightly stale)
- User profiles

**Performance over accuracy**

**Soft real-time requirements**

- Gaming leaderboards
- Trending topics
- Activity streams

### CAP Theorem states you can only have 2 of 3:

- **C**onsistency (all nodes see same data)
- **A**vailability (system always responds)
- **P**artition tolerance (system works despite network failures)

ACID databases typically chooses : **C**onsistency + **A**vailability (When no partitions)

ACID databases typically chooses : **C**onsistency + **P**artition tolerance (When partitions)

BASE databases typically chooses : **P**artition tolerance + **A**vailability

